I. Dokumentacja do wykładu. nr 2 - "Paradygmaty obiektowości"

II. Filip Florenc - grupa ABC, semestr III

III. Przedmiot - "Programowanie obiektowe"

IV. Opis zadania do realizacji

Do zrealizowania były następujące zadania:

- z pomocą dowolnego języka programowania napisz program, który po uruchomieniu:
  - wyświetli w kolejności alfabetycznej nazwy czterech filarów OOP,
  - wyświetli w kolejności losowej nazwy czterech filarów OOP,
  - w obu ww. przypadkach wskazane jest użycie wbudowanych w dany język programowania metod.

V. Technologie wykorzystane w zadaniu

- Java

VI. Realizacja zadania

1. Kod Javy (lub Pythona)

W zadaniu wykorzystano:

- Tablica i lista (`String[]` , `List<String>`)
- Metoda `Arrays.sort()`
- Metoda `Arrays.asList()`
- Metoda `Collections.shuffle()`
- Pętla `for-each` i `System.out.println()`

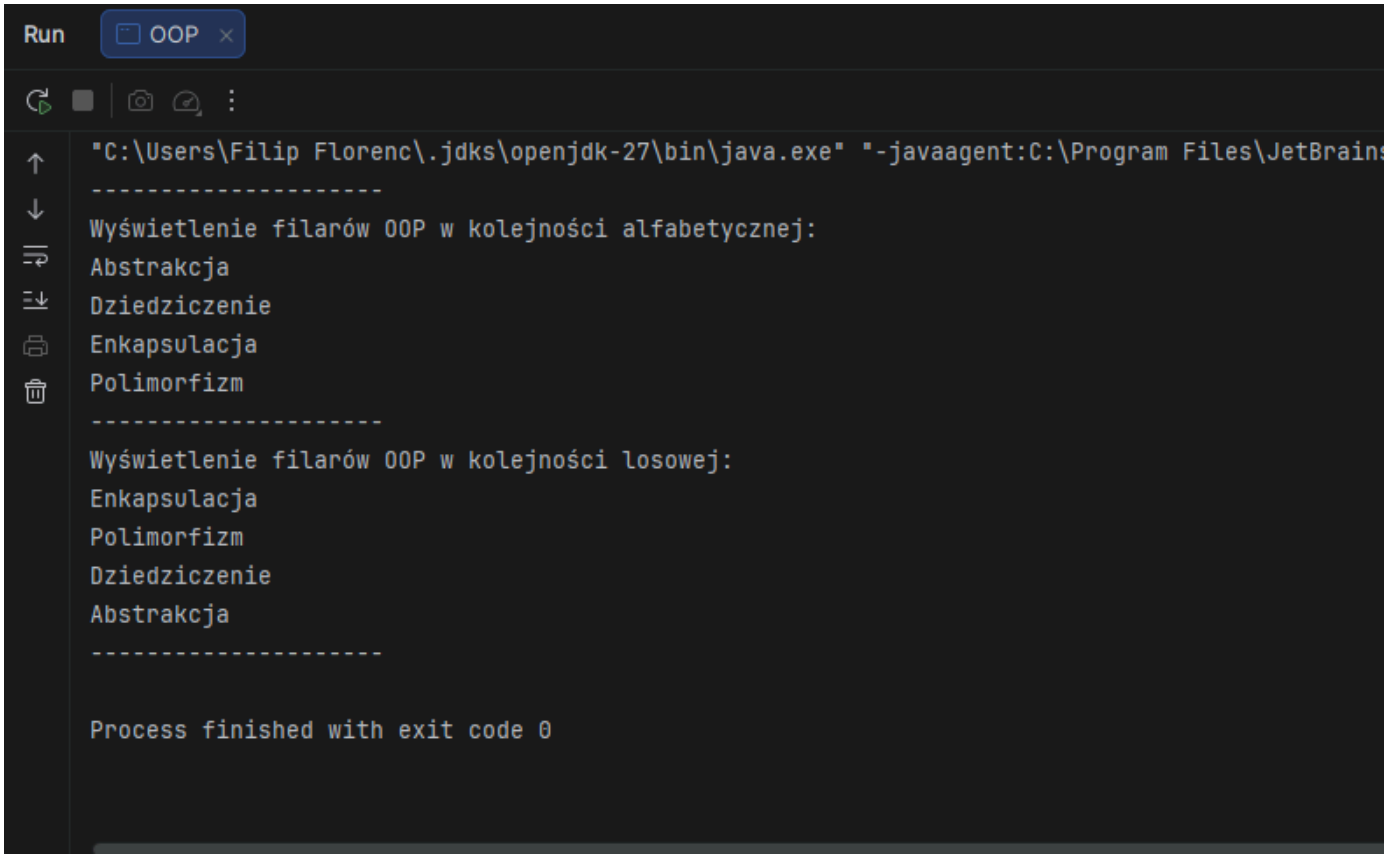
Kod wykorzystany do rozwiązania zadania (zadań):

```
    import java.util.Arrays;
import java.util.Collections;
import java.util.List;
public class OOP {
    public static void main(String[] args) {
```

```
        String[] filaryArray = {"Abstrakcja", "Enkapsulacja", "Dziedziczenie", "Polimorfizm"};

        Arrays.sort(filaryArray);
        System.out.println("---------------------");
        System.out.println("Wyświetlenie filarów OOP w kolejności alfabetycznej:");
        for (String filar : filaryArray) {
            System.out.println(filar);
        }
        System.out.println("---------------------");
        System.out.println("Wyświetlenie filarów OOP w kolejności losowej:");
        List<String> filaryLista = Arrays.asList(filaryArray);
        Collections.shuffle(filaryLista);
        for (String f : filaryLista)
        {
            System.out.println(f);
        }
        System.out.println("---------------------");
    }
}
```

2. Zrzuty ekranu pokazujące wynik działania aplikacji/skryptu:



3. Inne

Cztery filary OOP

> **Abstrakcja** - pokazywanie tylko najważniejszych informacji o obiekcie, bez konieczności poznawania tego, jak działa on od środka.

Enkapsulacja (hermetyzacja) - łączenie danych i metod w klasie oraz kontrolowanie dostępu do nich za pomocą modyfikatorów, np. private, protected i public.

Dziedziczenie - tworzenie nowych klas na podstawie już istniejących, dzięki czemu można wykorzystać ich pola i metody zamiast pisać wszystko od nowa.

Polimorfizm - możliwość wywołania tej samej metody na obiektach różnych klas, przy czym każdy z nich może działać w inny sposób. Dzięki temu można traktować różne obiekty jako obiekty tej samej klasy bazowej.